

Docker

docker compose

Rostagni Csaba

2025. szeptember 14.

Tartalom

1 Docker compose

Tartalom

- 1 Docker compose
 - YAML fájl
 - docker compose

YAML

- YAML Ain't Markup Language
- Kiterjesztés `.yaml` vagy `.yml` utóbbi a hivatalos
- Az egymásbaágyazást behúzással jelöli, mint a Python
 - **nem** használható a tabulátor karakter
 - gép által könnyen olvasható
 - emberi szemmel jól olvasható, átlátható

Linkek:

- <https://en.wikipedia.org/wiki/YAML>
- <https://yaml.org/>
- <https://yaml.org/refcard.html>

YAML példa

```
penztaros: Anna
kiallitva: 2020.09.01 11:05:32
termekek:
  - megnevezes: kenyér
    mennyiseg: 0.5
    mertkegyseg: kg
    ar: 280
  - megnevezes: tej
    mennyiseg: 1
    mertkegyseg: liter
    ar: 310
```

Egyszerű tömb

Python

```
["alma", "barack", "körte"]
```

YAML

```
- alma  
- barack  
- körte
```

YAML

```
- "alma"  
- "barack"  
- "körte"
```

- A tömb elemeit kötőjellel és egy szóközzel jelöljük
- A string jelölése opcionális
- Egy sorban is megadható, ahogy a python példán is látszik

Asszociatív tömb (map/dictionary)

Python

```
{"name": "Tóth Péter", "age": 21}
```

YAML

```
name: Tóth Péter  
age: 21
```

YAML

```
name: "Tóth Péter"  
age: 21
```

- Az asszociatív tömb kulcs-érték párait kettősponttal kell elválasztani
- A kettőspont után kell egy szóköz
- A string jelölése opcionális
- Egy sorban is megadható, ahogy a python példán is látszik

Tartalom

- 1 Docker compose
 - YAML fájl
 - docker compose

Docker compose

- A több container-ből álló alkalmazások kezelésére szolgáló eszköz
- Nagyban leegyszerűsíti az indítás, leállítás újraindítás folyamatát
- Meghatározható az indítás sorrendje
- A beállításokat a **Compose file** tartalmazza
 - Ez egy YAML fájl
 - Az alapértelmezett fájl `compose.yaml` az 1.28.6 verziótól
 - Kompatibilitási okokból elfogadja a `docker-compose.yaml` fájlt is, ez az elterjedtebb
- Összeállítható több fájlból is

Linkek:

- [docker compose – docs.docker.com](https://docs.docker.com/compose/)
- [compose fájl – docs.docker.com](https://docs.docker.com/compose/compose-file/)

docker compose parancsok

```
docker compose up
```

Terminal

- A szolgáltatásokat inicializálja a YAML fájl alapján, majd indítja a szolgáltatásokat

```
docker compose stop
```

Terminal

- Leállítja a szolgáltatásokat, de NEM törli

```
docker compose start
```

Terminal

- Elindítja a szolgáltatásokat

```
docker compose down
```

Terminal

- Leállítja a szolgáltatásokat, ÉS megszünteti a containereket

docker compose parancsok

Terminal

```
docker compose restart
```

- Újraindítja a szolgáltatásokat (stop + start)

Terminal

```
docker compose ps
```

- Kilistázza a containereket, amik az adott szolgáltatásokhoz tartoznak

Terminal

```
docker compose logs
```

- Kiírja logokat, amik az adott szolgáltatásokhoz tartoznak

docker compose V1 vs V2

```
docker-compose up
```

Terminal

- A V1 python script volt docker-compose néven
- **ma már elavultnak számít**

```
docker compose up
```

Terminal

- A V2 része a dockernek, nem külön script

Felső szintű bejegyzések

- `services` – Futtatandó szolgáltatások (container-ek és hozzá tartozó konfigurációs adatok)
- `networks` – A szolgáltatások a meghatározott hálózatokon kommunikálnak
 - Alapértelmezetten létrehoz egy `default` nevű hálózatot
- `volumes` – Tartós adattárolásra szolgáló kötetek (pl.: adatbázis)
- ...

Linkek:

- `compose file` – [docs.docker.com](https://docs.docker.com/compose/compose-file/)
- `Networking in Compose` – [docs.docker.com](https://docs.docker.com/compose/networking/)
- `Volumes top-level element` – [docs.docker.com](https://docs.docker.com/compose/volumes/)

nginx átírása compose fájlra

Terminal

```
docker run -d --name webservers -p 8080:80 \
-v "$(pwd)/www:/usr/share/nginx/html:ro" \
nginx
```

YAML

```
services:
  webservers:
    image: nginx
    ports:
      - 8080:80
    volumes:
      - ./www:/usr/share/nginx/html:ro
```

Terminal

```
docker compose up -d
```

nginx átírása compose fájlra

YAML

```
services:
  webserv:
    image: nginx
    ports:
      - 8080:80
    volumes:
      - ./www:/usr/share/nginx/html:ro
```

- Egyetlen szolgáltatás van `webserv`, ami az `nginx:latest` image-t használja
- A container `80`-as portja kívülről a `8080` porton érhető el
- Több port is lehetne, erre utal a többesszám `ports`, és utána a felsorolás. Hasonlóan lehetne több volume is felcsatolva
- Itt kivételesen lehetséges relatív hivatkozás, a `./` a `compose.yaml` fájlhoz képest viszonyít
- Amennyiben a mappa nem létezik létrehozza, hibás jogosultsággal

services.xxx.image

- Meghatározza, hogy a szolgáltatás melyik image-ből hozza létre a container-t
- Ha az image nincs, akkor kiadja a pull-t (ez beállítás függő)
- Elhagyható, ha a `build` szakasz definiálva van

```
services:
  database:
    image: mysql:8.0.39-debian
    ...
```

Dockerfile

- Szerencsére a kettőspont az image nevében nem okoz zavart, de a syntaxhighlighter megzavarodhat

services.xxx.container_name

- Meghatározza, hogy mi legyen a container neve
- Alapértelmezetten az alábbi séma szerint határozza meg a nevet
 - projekt/mappa -service- n ,
 - ahol az n a példány sorszáma, általában 1

```
services:  
  database:  
    container_name: adatbazis  
    ...
```

Dockerfile

services.xxx.build

Meghatározza, hogyan kell elkészíteni az image-t

```
build: ./database
```

Dockerfile

- Megadható egyszerű szöveges formátumban
- A példa a `./database/Dockerfile` alapján buildel

```
build:  
  context: ./database  
  dockerfile: mysql.Dockerfile
```

Dockerfile

- Meghatározható objektumként is
 - A `context` a környezet, ahol általában a Dockerfile van
 - Alapértelmezetten az aktuális mappa lesz `.`
 - A `dockerfile` az útvonal a context-hez képest
- A példa a `./database/mysql.Dockerfile` alapján buildel

services.xxx.command

Terminal

```
docker run -e MYSQL_ROOT_PASSWORD=password -d mysql:8 \
  --default-authentication-plugin=mysql_native_password
```

A CMD írható felül vele

Dockerfile

```
services:
  database:
    command: --default-authentication-plugin=mysql_native_password
    ...
```

- Amennyiben nincs kitöltve, vagy null az értéke, úgy a Dockerfile tartalma érvényesül
- Az üres string `""` és az üres tömb `[]` felülírja üressel
- A fenti példa úgy futtatja a mysql szerveret (`mysqld`), hogy átadja a megadott paramétert

services.xxx.restart

Meghatározza, hogy mi történjen, amikor leáll a container

- `no` – soha ne induljon újra (alapértelmezett)
- `always` – mindig induljon újra
- `unless-stopped` – csak akkor induljon újra, ha nem lett leállítva
- `on-failure` – csak akkor, ha hibakóddal állt le a container

Dockerfile

```
services:
  database:
    restart: unless-stopped
    ...
```

Linkek:

- Restart policy – [docs.docker.com](https://docs.docker.com/compose/reference/06/#restart-policy)
- restart – [docs.docker.com](https://docs.docker.com/compose/reference/06/#restart-policy)

services.xxx.environment

A container környezeti változóit állítja be

Dockerfile

```
services:
  database:
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: example
      ...
```

- Meghatározható kulcs-érték párokként
- Ilyenkor a kulcs és az érték között kettőspont szerepeljen!
- Minden logikai értéket idézőjelbe kell rakni, hogy a YAML értelmező ne alakítsa át logikai értéké
 - `true` , `false` , `yes` , `no`

services.xxx.environment

A container környezeti változóit állítja be

Dockerfile

```
services:
  database:
    environment:
      - MYSQL_ROOT_PASSWORD=root_password
      - MYSQL_DATABASE=example
      ...
```

- Meghatározható egyszerű tömbként
- Az egyenlőség előtt és után sincs szóköz